

Ejercicio Demostrativo

Sergio Sebastian Sandoval Ruiz

Tiempo estimado: 25 min

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Estructura de Datos - C]

1. Marco Formativo

1.1 Valores

Eficiencia	El estudiante evidencia este valor cuando elige implementar un BST o AVL en lugar de una lista enlazada para un problema donde las búsquedas son frecuentes, justificando la mejora de $O(n)$ a $O(\log n)$.
-------------------	---

1.2 Competencias

Tipo de Competencia	Redacción
Competencia General	Maneja grandes volúmenes de datos aplicando árboles para optimizar búsquedas.
Competencia Específica	Implementa un Árbol Binario de Búsqueda (BST) con las operaciones de inserción, búsqueda y eliminación; aplica los tres recorridos en profundidad (Pre-Orden, In-Orden y Post-Orden); e implementa un Árbol AVL identificando el factor de equilibrio y aplicando las cuatro rotaciones (RI, RD, RID, RDI) para mantener el balanceo automático tras cada operación.

2. Palabras Clave

Árbol: Estructura de datos no lineal y jerárquica compuesta por nodos conectados mediante aristas, donde un nodo especial llamado raíz no tiene padre y todos los demás tienen exactamente uno.

Nodo raíz: Primer nodo del árbol, único nodo sin padre. Es el punto de entrada a toda la estructura.

Nodo hoja: Nodo sin hijos. Siempre se encuentra en los extremos del árbol (nivel más profundo de su rama).

Nodo interior (rama): Nodo que no es raíz y tiene al menos un hijo. Conecta la raíz con las hojas.

Altura del árbol: Número máximo de niveles del árbol. Un árbol con solo la raíz tiene altura 1; un árbol vacío tiene altura 0.

Peso del árbol: Número total de nodos que contiene el árbol.

Árbol Binario de Búsqueda (BST): Árbol binario donde para cada nodo, todos los valores del subárbol izquierdo son menores y todos los del subárbol derecho son mayores. Permite búsquedas en $O(\log n)$ en el caso promedio.

Recorrido Pre-Orden: Visita los nodos en el orden Raíz → Izquierdo → Derecho. Útil para serializar la estructura del árbol.

Recorrido In-Orden: Visita los nodos en el orden Izquierdo → Raíz → Derecho. En un BST produce los elementos en orden ascendente.

Recorrido Post-Orden: Visita los nodos en el orden Izquierdo → Derecho → Raíz. Útil para liberar memoria o evaluar expresiones.

Árbol AVL: BST con la condición adicional de que la diferencia de alturas entre subárbol izquierdo y derecho de cualquier nodo (factor de equilibrio) no excede 1 en valor absoluto. Garantiza $O(\log n)$ en el peor caso.

Factor de equilibrio: Resultado de (altura subárbol derecho) - (altura subárbol izquierdo). Valores válidos: -1, 0, +1. Si es ≤ -2 o $\geq +2$, se requiere rotación.

Rotación simple (RI / RD): Corrección de desequilibrio cuando el nodo desbalanceado y su hijo tienen el mismo signo en el factor. Involucra solo 2 nodos.

Rotación doble (RID / RDI): Corrección de desequilibrio cuando el nodo desbalanceado y su hijo tienen signos opuestos en el factor. Involucra 3 nodos y es equivalente a dos rotaciones simples consecutivas.

3. Resumen

Este ejercicio demostrativo presenta de forma guiada la implementación de un Árbol Binario de Búsqueda (BST) en Java, incluyendo inserción, búsqueda y los tres recorridos en profundidad. Posteriormente se extiende el BST al Árbol AVL, demostrando cómo calcular el factor de equilibrio tras cada inserción y cómo aplicar las cuatro rotaciones posibles (RI, RD, RID, RDI) para restaurar el balanceo automáticamente.

La clase se orienta a que el estudiante comprenda la diferencia entre una estructura lineal y una jerárquica, identifique visualmente cada tipo de nodo (raíz, rama, hoja), trace los recorridos correctamente, y entienda por qué el AVL garantiza $O(\log n)$ incluso en el peor caso mientras que un BST degenerado puede llegar a $O(n)$.

4.Contenido

4.1 El ejercicio consiste en implementar desde cero en Java un BST con las operaciones de inserción, búsqueda y los tres recorridos (Pre-Orden, In-Orden, Post-Orden). Con la secuencia de inserción [50, 30, 70, 20, 40, 60, 80] se construirá el árbol en pizarrón y luego en código. Posteriormente se demostrará el Árbol AVL insertando la secuencia [10, 20, 30] para provocar un desequilibrio y aplicar la rotación RI, seguida de [30, 20, 10] para la rotación RD, [30, 10, 20] para la RID y [10, 30, 20] para la RDI.

4.2 OBJETIVO DE LA DEMOSTRACIÓN

Demostrar el procedimiento para implementar manualmente un Árbol Binario de Búsqueda y un Árbol AVL en Java, aplicando los tres recorridos en profundidad y las cuatro rotaciones de balanceo, con el fin de que el estudiante comprenda cómo la propiedad BST garantiza búsquedas ordenadas, identifique cuándo y qué rotación aplicar según el factor de equilibrio, y reconozca la ventaja del AVL sobre un BST sin balanceo en escenarios de peor caso.

4.3 RECURSOS NECESARIOS

- Software: Java JDK 11+, IDE (IntelliJ IDEA / Eclipse / VS Code)
 - Herramientas técnicas: Compilador javac, terminal/consola
 - Materiales de apoyo: Diapositivas Semana 7, pizarrón para dibujar los árboles paso a paso, herramienta online de visualización (<https://visualgo.net/en/bst>)
 - Archivos: Archivo base **NodoBST.java** con la estructura del nodo ya declarada
 - Configuración mínima: Java 11+, 4 GB RAM, proyector
-

4.4 DESARROLLO PASO A PASO

Paso 1 — Terminología y diferencia Lineal vs Jerárquico (10 min)

Dibujar en pizarrón un árbol con 15 nodos (como el de la diapositiva) e identificar en voz alta con los estudiantes: la raíz, las hojas, los nodos internos, la altura y el peso. Comparar con una lista enlazada: en la lista para encontrar un elemento se recorren hasta n nodos; en el árbol se descartan subárboles completos en cada paso.

Pregunta al estudiante: En el árbol de las diapositivas con altura 4 y peso 15, ¿cuántos nodos como máximo debo revisar para encontrar un elemento con búsqueda binaria? (Respuesta: 4, la altura del árbol).

Paso 2 — Implementación del BST: Inserción (20 min)

Construir el árbol [50, 30, 70, 20, 40, 60, 80] dibujándolo en pizarrón antes de codificar. Luego implementar:

java

Java

```
class NodoBST {  
    int dato;  
    NodoBST izquierdo, derecho;  
    NodoBST(int d) { dato = d; }  
}  
  
class BST {  
    NodoBST raiz = null;
```

```
NodoBST insertar(NodoBST nodo, int valor) {  
    if (nodo == null) return new NodoBST(valor);  
    if (valor < nodo.dato)  
        nodo.izquierdo = insertar(nodo.izquierdo,  
valor);  
    else if (valor > nodo.dato)  
        nodo.derecho = insertar(nodo.derecho, valor);  
    return nodo; // valor duplicado: ignorar  
}  
  
boolean buscar(NodoBST nodo, int valor) {  
    if (nodo == null) return false;  
    if (valor == nodo.dato) return true;  
    if (valor < nodo.dato) return  
buscar(nodo.izquierdo, valor);  
    return buscar(nodo.derecho, valor);  
}  
}
```

Qué observar: La inserción es recursiva. Cada llamada descarta la mitad del árbol según la comparación, llegando al lugar correcto sin recorrer toda la estructura.

Paso 3 — Los tres recorridos sobre el BST (15 min)

Implementar los tres recorridos y trazar manualmente el resultado sobre el árbol del paso anterior:

java

Java

// Pre-Orden: Raíz → Izquierdo → Derecho

```
void preOrden(NodoBST nodo) {  
    if (nodo == null) return;  
    System.out.print(nodo.dato + " ");  
    preOrden(nodo.izquierdo);  
    preOrden(nodo.derecho);  
}
```

// In-Orden: Izquierdo → Raíz → Derecho

```
void inOrden(NodoBST nodo) {  
    if (nodo == null) return;  
    inOrden(nodo.izquierdo);  
    System.out.print(nodo.dato + " ");  
    inOrden(nodo.derecho);  
}
```

// Post-Orden: Izquierdo → Derecho → Raíz

```

void postOrden(NodoBST nodo) {
    if (nodo == null) return;
    postOrden(nodo.izquierdo);
    postOrden(nodo.derecho);
    System.out.print(nodo.dato + " ");
}

```

Resultados esperados con [50, 30, 70, 20, 40, 60, 80]:

- Pre-Orden: 50 30 20 40 70 60 80
- In-Orden: 20 30 40 50 60 70 80 ← siempre ordenado en un BST
- Post-Orden: 20 40 30 60 80 70 50

Qué observar: In-Orden de un BST siempre produce los elementos en orden ascendente. Esta propiedad es la base de muchos algoritmos de ordenamiento basados en árboles.

Paso 4 — Árbol AVL: Factor de equilibrio y rotaciones (25 min)

Explicar la fórmula: $\text{Factor} = \text{altura}(\text{subárbol derecho}) - \text{altura}(\text{subárbol izquierdo})$. Valores aceptables: -1, 0, +1. Si el factor es ≤ -2 o $\geq +2$, se rompe el AVL y hay que rotar.

Demostrar los 4 casos con secuencias simples:

java

Java

```

int altura(NodoBST n) {
    if (n == null) return 0;
}

```



```
        return 1 + Math.max(altura(n.izquierdo),
        altura(n.derecho));
    }

    int factor(NodoBST n) {
        return altura(n.derecho) - altura(n.izquierdo);
    }

    // Rotación Simple Izquierda (RI) - Factor >= +2 y hijo
    // derecho Factor >= 0
    NodoBST rotacionIzquierda(NodoBST x) {
        NodoBST y = x.derecho;
        x.derecho = y.izquierdo;
        y.izquierdo = x;
        return y; // y es la nueva raíz
    }

    // Rotación Simple Derecha (RD) - Factor <= -2 y hijo
    // izquierdo Factor <= 0
    NodoBST rotacionDerecha(NodoBST y) {
        NodoBST x = y.izquierdo;
        y.izquierdo = x.derecho;
```

```

    x.derecho = y;

    return x; // x es la nueva raíz
}

// RID - Factor <= -2 y hijo izquierdo Factor > 0
NodoBST rotacionIzqDer(NodoBST nodo) {
    nodo.izquierdo = rotacionIzquierda(nodo.izquierdo);
    return rotacionDerecha(nodo);
}

// RDI - Factor >= +2 y hijo derecho Factor < 0
NodoBST rotacionDerIzq(NodoBST nodo) {
    nodo.derecho = rotacionDerecha(nodo.derecho);
    return rotacionIzquierda(nodo);
}

```

Casos de demostración en pizarrón:

Secuencia insertada	Desequilibrio	Rotación aplicada	Árbol resultante
10 → 20 → 30	Nodo 10 factor +2, hijo 20 factor +1 (mismo signo)	RI en nodo 10	Raíz: 20, hijo izq: 10, hijo der: 30

30 → 20 → 10	Nodo 30 factor -2, hijo 20 factor -1 (mismo signo)	RD en nodo 30	Raíz: 20, hijo izq: 10, hijo der: 30
30 → 10 → 20	Nodo 30 factor -2, hijo 10 factor +1 (signo opuesto)	RID en nodo 30	Raíz: 20, hijo izq: 10, hijo der: 30
10 → 30 → 20	Nodo 10 factor +2, hijo 30 factor -1 (signo opuesto)	RDI en nodo 10	Raíz: 20, hijo izq: 10, hijo der: 30

Observaciones Importantes

- El **BST degenerado** ocurre cuando se insertan elementos en orden estrictamente ascendente o descendente. Ejemplo: insertar [10, 20, 30, 40, 50] produce un árbol con altura 5 y búsquedas $O(n)$, igual que una lista enlazada. El AVL previene exactamente esto.
- En el **recorrido In-Orden**, el único cambio respecto a Pre/Post es el orden de la línea `System.out.print`. Los estudiantes suelen confundirse memorizando los nombres; es mejor recordar **In = en medio** (Izq → **Raíz** → Der).
- Las **rotaciones simples** (RI, RD) mueven 2 nodos. Las **rotaciones dobles** (RID, RDI) son en realidad dos rotaciones simples consecutivas aplicadas desde abajo hacia arriba.
- La regla para elegir rotación es: mirar el signo del factor del nodo desbalanceado y el signo del factor de su hijo problemático. **Mismo signo** → **rotación simple**. **Signo opuesto** → **rotación doble**.

4.5 Tabla Comparativa — Pila vs Cola

TABLA COMPARATIVA

Criterio	Lista Enlazada	BST (promedio)	BST (peor caso)	AVL (siempre)
Búsqueda	$O(n)$	$O(\log n)$	$O(n)$	$O(\log n)$
Inserción	$O(1)$ al inicio	$O(\log n)$	$O(n)$	$O(\log n)$

Eliminación	$O(n)$	$O(\log n)$	$O(n)$	$O(\log n)$
Estructura	Lineal	Jerárquica	Jerárquica (degenerada)	Jerárquica balanceada
Garantía de $O(\log n)$	No	No	No	Sí siempre

4.6 Resultado Esperado

Operación	Resultado
Insertar [50, 30, 70, 20, 40, 60, 80] en BST	Árbol balanceado con raíz 50, altura 3, peso 7
Pre-Orden sobre ese BST	50 30 20 40 70 60 80
In-Orden sobre ese BST	20 30 40 50 60 70 80 (ordenado)
Post-Orden sobre ese BST	20 40 30 60 80 70 50
Insertar [10, 20, 30] en AVL	Rotación RI aplicada en nodo 10 → raíz queda en 20
Insertar [30, 10, 20] en AVL	Rotación RID aplicada en nodo 30 → raíz queda en 20

4.7 Aplicación Práctica

Los algoritmos de ordenamiento son la base de casi cualquier sistema de información: ordenar registros de pacientes, productos en un catálogo, resultados de búsqueda. La elección correcta impacta directamente el rendimiento; por eso el curso de cátedra mostró la tabla de complejidades comparativas.

La Matriz Dispersa es fundamental en el proyecto del curso (Cinema/MedTrack): el mapa de asientos por función es una matriz donde la mayoría de celdas están vacías. Implementarla como matriz densa (arreglo 2D completo) desperdicia memoria; implementarla como Matriz Dispersa con cabeceras solo almacena los asientos ocupados o reservados.

5. Aconsejando al siguiente Tutor para la realización de ejemplos

Dificultades encontradas:

- Los estudiantes confunden cuándo aplicar RI vs RD: la rotación se nombra hacia donde "cae" la raíz desbalanceada, no hacia donde está el exceso de peso. Es más fácil recordarlo como: "la raíz baja hacia la izquierda → RD; la raíz baja hacia la derecha → RI."
- Las rotaciones dobles (RID y RDI) son el punto de mayor confusión. Hay que enfatizar que son dos rotaciones simples: primero se rota el hijo, luego se rota la raíz desbalanceada.
- El recorrido Post-Orden es el más difícil de trazar manualmente porque la raíz se visita al final, lo que va contra la intuición.

Recomendaciones:

- Usar el visualizador online (<https://visualgo.net/en/bst>) en tiempo real durante la demostración. Los estudiantes comprenden mucho mejor las rotaciones cuando las ven animadas.
- Dibujar SIEMPRE el árbol en pizarrón antes de escribir código. El árbol es un concepto visual; entenderlo en papel primero ahorra el doble de tiempo en debugging.
- Para las rotaciones, preparar 4 árboles de 3 nodos pre-dibujados (uno por cada caso) y mostrarlos antes de mostrar el código de la rotación.
- Demostrar el BST degenerado insertando [1,2,3,4,5] en secuencia para que el estudiante vea visualmente por qué el AVL es necesario.

Puntos de mayor dificultad:

- Identificar correctamente el tipo de rotación a partir del factor del nodo y el factor de su hijo.

- El recorrido Post-Orden trazado manualmente.
- Entender que en la rotación el nodo que "sube" se convierte en la nueva raíz del subárbol y que el puntero del padre también debe actualizarse.

6. Referencias

1. GeeksforGeeks. (2023). Binary Search Tree.
<https://www.geeksforgeeks.org/binary-search-tree-data-structure/>
2. GeeksforGeeks. (2023). AVL Tree.
<https://www.geeksforgeeks.org/avl-tree-set-1-insertion/>
3. GeeksforGeeks. (2023). Tree Traversals (Inorder, Preorder, Postorder).
<https://www.geeksforgeeks.org/tree-traversals-inorder-preorder-and-postorder/>
4. Visualizador interactivo de árboles: <https://visualgo.net/en/bst>
5. Cormen, T. H. et al. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Cap. 12: Árboles Binarios de Búsqueda. Cap. 13: Árboles Rojo-Negro (base teórica de árboles balanceados).
6. Weiss, M. A. (2013). *Data Structures and Algorithm Analysis in Java* (3rd ed.). Pearson. Cap. 4: Árboles.
7. Material de cátedra: Diapositivas Semana 7, EDD [B] — Árboles BST y AVL. USAC, 2026.
- 8.